

SOLUTION:

07 September 2026 • Architecture.md • City-Wide AI Engine

Stack PREFINAL.pptx + main.py + api.py	Source docs/ • Markdown → PDF	Team SIH 26127 • Prefinal
--	---	-------------------------------------

SIH PROBLEM STATEMENT- 26127

title : City-Wide AI Engine for Multi-Camera ANPR Trajectory Tracking and Urban Traffic Analytics

description: • Background Modern urban centers deploy vast networks of CCTV and Automatic Number Plate Recognition(ANPR) cameras to manage traffic, enforce traffic laws, and maintain public security. However,most existing systems process these feeds in isolated silos, performing basic license plate detection without effectively linking data across space and time. This lack of integration prevents city authorities from automatically tracking high-interest vehicles across different sectors and limits their ability to extract macro-level traffic movement trends from the existing camera infrastructure. • Description The objective is to develop a robust, centralized AI software platform that processes multicamera feeds across a city-wide ANPR network to accomplish three core functionalities. First, the platform must feature a High-Accuracy ANPR and OCR Engine, which utilizes an advanced Optical Character Recognition model capable of achieving greater than 90% accuracy across diverse realworld conditions such as varying lighting, poor weather, angled shots, motion blur, and dirty or damaged license plates. Second, it requires a Single Plate Trajectory Tracking module to build a spatial-temporal tracking system capable of reconstructing the complete travel trajectory of any specific vehicle plate across the entire city network. This system will map a vehicle's movement history, timestamps, direction, and route on a GIS map using inputs from geographically distributed ANPR cameras. Third, the system must perform Macro Traffic Flow and Movement Analytics by analyzing aggregated camera data to compute and visualize general city-wide traffic dynamics. This includes measuring traffic density, identifying origin-destination patterns,detecting congestion bottlenecks, and providing real-time heatmaps of city traffic movement. • Expected Solution The expected solution is a scalable, enterprise-grade software platform equipped with four key components. It will feature a High-Precision OCR Module powered by a deep-learning model exceeding 90% recognition accuracy for license plates in multi-lane traffic streams. It will include a Trajectory Reconstruction Engine providing a query-based tracking interface that plots a vehicle's historical path chronologically across the city map with accurate timestamps and camera locations. Furthermore, it will integrate a City Traffic Analytics Dashboard to serve as a centralized, GIS-integrated web platform displaying heatmaps, average vehicle speeds, route densities, and traffic flow trends across all camera nodes. Finally, the platform will incorporate an Alert System capable of flagging blacklisted vehicles and suspicious route anomalies in real time.

Start with a rock-solid **single-camera perception pipeline** that turns raw CCTV frames into structured vehicle observations.

The important architectural insight is:

ANPR is not the product. Vehicle observations + persistent vehicle identity + spatial-temporal association are the foundation of the product.

For each detected vehicle, your Python processor should ideally emit something like:

```
{
  "camera_id": "CAM_042",
  "timestamp": "2026-08-27T05:30:12.421Z",
  "latitude": 90,
  "longitude": 90,

  "plate": {
    "text": "OD02AB1234",
    "confidence": 0.96
  },

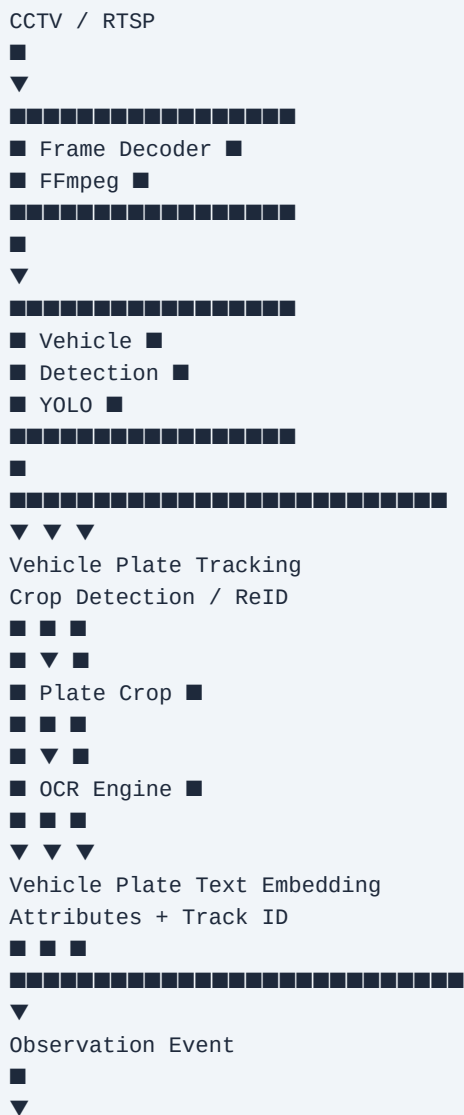
  "vehicle": {
    "type": "car",
    "color": "white",
    "make": "Maruti Suzuki",
    "model": "Swift",
```

```
"confidence": 0.81,
"wheelers": 4
},
"motion": {
"speed_kmh": 42.7,
"direction": 87.3
},
"tracking": {
"local_track_id": 182,
"embedding": [0.021, -0.183, "..."]
},
"bbox": [412, 217, 681, 486]
}
```

But don't build this as one giant neural network. Build a **pipeline of specialized models**.

1. Your first MOAT architecture

I'd build this:





This separation is extremely important.

If your OCR model improves tomorrow, you shouldn't have to retrain your vehicle detector.

If you add motorcycles later, you shouldn't rewrite tracking.

If you replace YOLO with another detector, the rest of the system should continue working.

2. Start with these 6 capabilities

For your MVP, I'd target:

Capability	First implementation
Vehicle detection	YOLO
Vehicle tracking	ByteTrack
Plate detection	YOLO
Plate OCR	PaddleOCR / custom OCR
Vehicle embedding	Vehicle Re-ID model
Speed	Camera calibration + tracking

Then later:

```
vehicle color
vehicle make
vehicle model
vehicle type
license plate
plate country/state
direction
speed
lane
embedding
track ID
timestamp
camera ID
```

3. Vehicle detection

Don't detect plates first.

Detect vehicles first.

For example:

[illegible]

Use something like:

```
pip install ultralytics
```

Then:

```
from ultralytics import YOLO

model = YOLO("yolo11m.pt")

results = model(frame)

for result in results:
    for box in result.boxes:
        cls = int(box.cls[0])
        confidence = float(box.conf[0])

        x1, y1, x2, y2 = box.xyxy[0].tolist()

        print({
            "class": cls,
            "confidence": confidence,
            "bbox": [x1, y1, x2, y2]
        })
```

For your real system, you'll probably eventually train/fine-tune on **Indian traffic CCTV**.

That dataset is going to matter enormously.

4. Tracking is more important than you initially think

Suppose:

```
frame 001 → Car
frame 002 → Car
frame 003 → Car
frame 004 → Car
```

The detector doesn't inherently know that these are the same car.

Tracking gives you:

```
Car
■■■ track_id = 182
■■■ frame 001
■■■ frame 002
■■■ frame 003
■■■ frame 004
```

Use **ByteTrack** initially.

Conceptually:

```
from ultralytics import YOLO

model = YOLO("vehicle.pt")

results = model.track(
    source="traffic.mp4",
    tracker="bytetrack.yaml",
    persist=True
)

for result in results:
    boxes = result.boxes

    if boxes.id is None:
        continue

    track_ids = boxes.id.int().tolist()

    for track_id, box in zip(track_ids, boxes):
        print(track_id, box.xyxy)
```

Now you can accumulate observations:

```
track_id 182

05:30:10 → (412,217)
05:30:11 → (421,219)
05:30:12 → (430,222)
05:30:13 → (441,224)
```

That gives you the foundation for speed and trajectory.

5. License plate detection

This should be a **separate model**.

Don't simply crop the lower half of the vehicle.

Traffic cameras can have:

```
front plates
rear plates
angled plates
motorcycle plates
occluded plates
multiple vehicles
```

Train something like:

```
Input
↓
Plate detector
↓
[x1,y1,x2,y2]
↓
Perspective correction
↓
OCR
```

For example:

```
plate_model = YOLO("license_plate.pt")

results = plate_model(vehicle_crop)

for result in results:
    for box in result.bboxes:
        plate_crop = crop(
            vehicle_crop,
            box.xyxy[0]
        )
```

Your dataset here is extremely valuable.

6. OCR

For the first prototype, don't immediately train your own OCR model.

Try:

PaddleOCR

or

EasyOCR

PaddleOCR would be my first choice for this project.

Pipeline:

```
plate crop
■
▼
perspective correction
■
▼
image enhancement
■
▼
OCR
■
▼
OD02AB1234
```

But here's a very important senior-level trick:

Don't OCR every frame.

Suppose your camera runs at:

```
30 FPS
```

A car stays visible for:

```
3 seconds
```

You now have:

```
90 OCR operations
```

for the same vehicle.

That's stupidly expensive.

Instead:

```
Track 182
■
■■■ frame 1
■■■ frame 5 → OCR
■■■ frame 10
■■■ frame 15 → OCR
■■■ frame 20
■■■ frame 25 → OCR
```

Then combine:

```
OD02AB1234 0.91
OD02AB1234 0.97
OD02AB1234 0.94
```

and select the highest-confidence / consensus result.

This is **temporal OCR aggregation**.

It can dramatically improve practical recognition.

7. Vehicle color

This is relatively easy.

Start with a classifier:

```
vehicle crop
↓
Vehicle Color Classifier
↓
white
```

Classes:

```
black
white
silver
gray
red
blue
green
```



```
yellow
orange
brown
other
```

Don't over-engineer this initially.

You could even start with a pretrained vision model and fine-tune it later.

8. Vehicle type / wheeler

Your detector can classify:

```
car
motorcycle
bus
truck
auto-rickshaw
van
bicycle
```

Then derive:

```
wheeler = {
  "motorcycle": 2,
  "bicycle": 2,
  "car": 4,
  "bus": 6,
  "truck": 6
}
```

But be careful:

wheeler count is not a reliable physical property from a single camera view.

A better semantic field is:

```
{
  "vehicle_type": "motorcycle",
  "axle_estimate": null,
  "wheeler": 2
}
```

rather than pretending your model knows the exact wheel count.

9. Vehicle make/model

This is where the problem gets substantially harder.

You want:

```
Maruti Suzuki Swift
```

rather than:

```
car
```

This should be a dedicated **vehicle make/model classifier**.

Pipeline:

```
vehicle crop
■
▼
Vehicle embedding
■
■■■ similarity search
■
▼
Make/model classifier
■
▼
Maruti Suzuki Swift
```

Initially I'd support maybe:

```
Maruti
Hyundai
Tata
Mahindra
Toyota
Honda
Kia
MG
Renault
Volkswagen
```

and expand later.

10. The REALLY important part: vehicle embeddings

This is probably the most strategically important part of your entire MOAT.

You don't want:

```
plate = OD02AB1234
```

to be your only identity.

You want:

```
vehicle
■
■■■ plate
■■■ appearance
■■■ color
■■■ make
■■■ model
■■■ embedding
```

For example:

```
{
  "track_id": 182,

  "plate": "OD02AB1234",
```

SOLUTION:

```
"embedding": [  
  0.12,  
  -0.31,  
  0.44,  
  ...  
]  
}
```

The embedding represents the vehicle's visual appearance.

Then camera A sees:

```
white Swift  
embedding A
```

Camera B later sees:

```
white Swift  
embedding B
```

You calculate:

```
cosine_similarity(A, B)
```

and get:

```
0.94
```

Potentially the same vehicle.

That's **vehicle re-identification (Re-ID)**.

11. Plate + Re-ID is MUCH stronger than either alone

Imagine:

Camera A

```
Plate: OD02AB1234  
Color: White  
Model: Swift  
Embedding: E1
```

Camera B

```
Plate: OD02AB1234  
Color: White  
Model: Swift  
Embedding: E2
```

Now your confidence becomes much stronger:

```
plate similarity 1.00  
appearance similarity 0.94  
color similarity 1.00  
model similarity 1.00  
time consistency 0.97  
route consistency 0.91
```

You can eventually build a matching score:

```
identity_score =
0.40 * plate_score
+ 0.25 * embedding_score
+ 0.10 * color_score
+ 0.10 * model_score
+ 0.15 * temporal/spatial_score
```

That is where your eventual **trajectory engine** comes from.

12. Speed estimation

This is another place where people build a misleading demo.

You cannot reliably calculate:

pixels moved / time

and call that km/h .

You need camera calibration.

For a fixed CCTV camera:

Define two reference zones:

```
line A
line B
```

Known real-world distance:

20 meters

Vehicle:

line A $\rightarrow$ 1.8 sec $\rightarrow$ line B

Then:

```
speed = distance / time

20 / 1.8 = 11.11 m/s

11.11 x 3.6 = 40 km/h
```

For a prototype, this is excellent.

Later you can use:

- camera calibration
- homography
- road-plane projection
- optical flow
- multi-camera geometry

13. Direction

Once you have track coordinates:

```
t0 → (400, 300)
t1 → (410, 302)
t2 → (421, 305)
t3 → (433, 308)
```

Calculate:

```
dx = x2 - x1
dy = y2 - y1
```

and:

```
import math

angle = math.degrees(math.atan2(dy, dx))
```

You can convert that into:

```
N
NE
E
SE
S
SW
W
NW
```

For GIS later, you'd want the **actual road heading**, not merely image-space heading.

14. Your Python processor should become a service

I would structure your Python project roughly like:

```
ai-engine/
├──
├── app/
│   └── main.py
├──
├── pipeline/
│   └── detector.py
```

```

■ ■ ■■ tracker.py
■ ■ ■■ plate_detector.py
■ ■ ■■ ocr.py
■ ■ ■■ vehicle_classifier.py
■ ■ ■■ vehicle_reid.py
■ ■ ■■ speed.py
■ ■
■ ■■ models/
■ ■ ■■ vehicle/
■ ■ ■■ plate/
■ ■ ■■ ocr/
■ ■ ■■ reid/
■ ■
■ ■■ schemas/
■ ■ ■■ observation.py
■ ■
■ ■■ utils/
■ ■■ image.py
■ ■■ geometry.py
■
■■■ tests/
■
■■■ requirements.txt
■■■ Dockerfile

```

And the main pipeline:

```

def process_frame(frame, camera):

    vehicles = vehicle_detector.detect(frame)

    tracks = tracker.update(vehicles)

    observations = []

    for track in tracks:

        vehicle_crop = crop(frame, track.bbox)

        plate = plate_detector.detect(vehicle_crop)

        plate_text = None

        if plate:
            plate_crop = crop(vehicle_crop, plate.bbox)
            plate_text = ocr.recognize(plate_crop)

        embedding = reid.extract(vehicle_crop)

        attributes = vehicle_classifier.predict(vehicle_crop)

        speed = speed_estimator.update(track)

        observations.append({
            "camera_id": camera.id,
            "track_id": track.id,
            "plate": plate_text,
            "embedding": embedding,
            "attributes": attributes,
            "speed": speed,

```

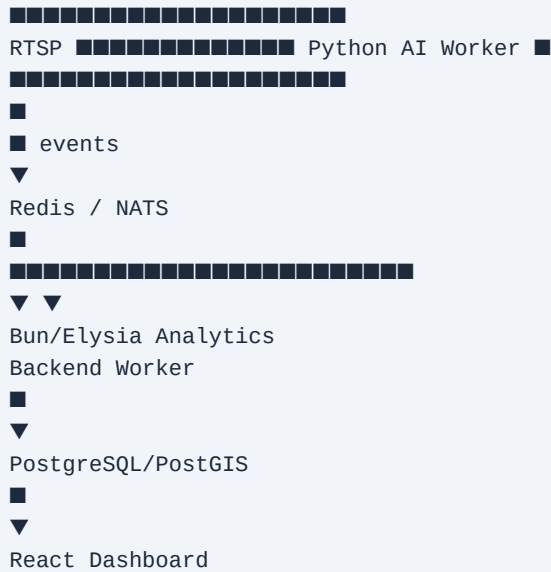
```
"timestamp": timestamp()
})

return observations
```

15. Then expose it to your Node/Bun ecosystem

Don't make your Bun backend run Python ML directly.

I'd use:



For your stack, **Redis Streams** or **NATS** would be a good starting point.

The Python side produces:

```
{
  "event": "vehicle.detected",
  "cameraId": "CAM_042",
  "timestamp": 1756278612421,
  "trackId": 182,
  "plate": "OD02AB1234",
  "plateConfidence": 0.96,
  "vehicleType": "car",
  "color": "white",
  "make": "Maruti Suzuki",
  "model": "Swift",
  "speed": 42.7,
  "direction": 87,
  "embedding": [...]
}
```

Your Bun backend consumes it.

16. Don't store embeddings directly in PostgreSQL as your first design

Eventually you'll want something like:

```
PostgreSQL + PostGIS
+
pgvector
```

Then:

```
SELECT *
FROM vehicle_observations
ORDER BY embedding <=> $embedding
LIMIT 10;
```

That gives you:

"Find vehicles visually similar to this vehicle."

This becomes extremely powerful.

Your eventual system could answer:

```
Find all observations of vehicles visually
similar to this vehicle between 10:00 and 12:00.
```

That's much closer to a serious **vehicle intelligence platform** than simply building ANPR.

17. The database model I'd eventually aim for

Something along these lines:

```
Camera
■
■■■ id
■■■ location
■■■ latitude
■■■ longitude
■■■ heading
■■■ calibration

VehicleObservation
■
■■■ id
■■■ camera_id
■■■ timestamp
■■■ track_id
■■■ plate_text
■■■ plate_confidence
■■■ vehicle_type
■■■ color
■■■ make
■■■ model
■■■ speed
■■■ direction
■■■ bbox
■■■ embedding

VehicleIdentity
■
■■■ id
■■■ canonical_plate
```



```

    canonical_embedding
    vehicle_type
    color
    make
    model

VehicleTrajectory
  vehicle_id
  camera_id
  timestamp
  latitude
  longitude
  speed
  direction

```

But **don't implement VehicleIdentity yet**.

That's phase 2.

First prove that the observation pipeline works.

18. Your actual MOAT milestone

I would define your first demo as:

```

CCTV VIDEO
  Vehicle Detection
    ByteTrack
    Plate Detection Vehicle Crop
    OCR Color
    Make/Model
    Embedding
  Vehicle Event
  React live dashboard

```

And the UI shows:

```

CAMERA CAM-042
[LIVE CCTV VIDEO]

```



```
0D02AB1234
```

An enterprise system shouldn't simply say:

```
OCR result = truth
```

It should reason:

```
Frame 1 → 0D02AB1234 (0.82)
Frame 2 → 0D02AB123B (0.71)
Frame 3 → 0D02AB1234 (0.97)
Frame 4 → 0D02AB1234 (0.95)
```

↓

```
Consensus → 0D02AB1234
```

That's the kind of engineering that will make your system robust rather than merely impressive in a demo.

Recommended stack for your MOAT

Python

- PyTorch
- Ultralytics/YOLO for initial detection
- ByteTrack for tracking
- PaddleOCR for initial OCR
- OpenCV for image processing + geometry
- ONNX Runtime/TensorRT later for production inference
- a Vehicle Re-ID model for embeddings
- FastAPI/gRPC for service boundaries

Infrastructure

- FFmpeg/GStreamer for RTSP
- Redis Streams or NATS for events
- PostgreSQL + PostGIS
- pgvector for embeddings
- Bun/Elysia for your application API
- React + MapLibre/Mapbox for GIS

And one thing I'd strongly recommend: **don't train everything from scratch**. Your competitive advantage is going to come from the *data pipeline*, *Indian traffic dataset*, *temporal OCR*, *vehicle Re-ID*, *camera calibration*, *cross-camera association*, and *trajectory algorithms*, not from reinventing YOLO or OCR. Start with pretrained models and spend your effort collecting and labeling the failure cases from your actual CCTV environment.